

Practice Set: Constants in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define what a **Constant** is in C++. Compare Variables and Constants across four criteria: Mutability, Memory State, C++ Code Syntax, and Re-assignment behavior.

Question 2: Outline the complete **Classification Hierarchy** of C++ Constants, categorizing them into Primary and Secondary constants along with their respective sub-categories.

Question 3: Describe the four core categories of **Primary Constants (Literals)** in C++ and provide code examples for each.

Question 4: Explain and compare the modern C++ constant evaluation keywords: `const`, `constexpr` (C++11), and `constexpr` (C++20).

Question 5: Detail the difference in RAM memory allocation between a runtime variable (e.g., `int X = 10;`) and a compile-time constant (e.g., `constexpr int Y = 10;`).

Question 6: What are **Secondary Constants** in C++? List the five derived/compound structures categorized under secondary constants.

Question 7: Explain the mechanics of **Pointer Constness** in C++. Differentiate between `const int* ptr`, `int* const ptr`, and `const int* const ptr` using the "right-to-left" reading rule.

Question 8: Describe how C-style string literals are represented in memory, including the role of the null-terminator. Name the modern C++ string abstractions recommended for type-safe execution.

Question 9: Explain the memory structure of a C++ struct versus a raw C union. What occurs when a union member is accessed after another member was written?

Question 10: Write a complete C++ program that demonstrates a `constexpr` integer constant, a `std::string` constant, a pointer to constant data, and an instantiated constant structure (struct).

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): What error occurs if a developer declares a `const` or `constexpr` variable without providing an initial value upon declaration?

Question 12 (Viva): Why is constexpr preferred over const for values known at compile time?

Question 13 (Viva): How does a C++20 constexpr immediate function differ from a C++11 constexpr function?

Question 14 (Viva): Using the "right-to-left rule," explain what const int* const ptr signifies.

Question 15 (Viva): If a pointer is declared as const int* ptr = &value;, can the value of value be updated directly through value = 20;?

Question 16 (Viva): What is the hidden character that marks the end of a C-style string literal, and what byte value does it store?

Question 17 (Viva): Which modern type-safe C++ container is recommended as an alternative to raw C unions?

Question 18 (Viva): Which modern C++ sequence container is recommended to replace raw C-style arrays?

Question 19 (Viva): What is the default member access specifier in a C++ struct compared to a C++ class?

Question 20 (Viva): How does compile-time constant evaluation reduce runtime CPU overhead?

Answers & Explanations

Answer : 01

A **Constant** is an immutable value that remains completely fixed throughout program execution, locked in memory or evaluated during compilation.

Comparison Matrix:

Mutability: Variables can change during execution; Constants cannot.

Memory State: Variable contents vary; Constant contents remain strictly locked.

C++ Syntax: Variables use `int age = 20;`; Constants use `const float PI = 3.14f;` or `constexpr double PI = 3.14159;`.

Re-assignment: age = 21; is allowed; PI = 5.0f; triggers a compile error.

Answer : 02

Primary Constants (Fundamental Singular Values):

Integer Literals (e.g., 10, -15)

Real / Floating-Point Literals (e.g., 3.14, -0.01)

Character Literals (e.g., 'A', '7')

Boolean Literals (true, false)

Secondary Constants (Derived / Compound Structures):

Arrays (std::array, C-style)

Strings (std::string, string literals)

Pointers (const int*, int* const)

Structures & Classes (struct, class)

Unions (std::variant, union)

Answer : 03

Integer Constants: Whole numbers without decimal parts (e.g., constexpr int attempts = 3;).

Real (Floating-Point) Constants: Decimal numbers containing fractional components (e.g., constexpr double pi = 3.14159;).

Character Constants: Single characters enclosed in single quotes (e.g., constexpr char grade = 'A';).

Boolean Constants: Built-in truth values true or false (e.g., constexpr bool isReady = true;).

Answer : 04

const: Evaluated at runtime or compile-time; guarantees read-only access after initialization.

constexpr (C++11): Guarantees compile-time evaluation, allowing constants to be directly embedded into program assembly/binary instructions.

constexpr (C++20): Specifies immediate functions that *must* evaluate at compile-time and produce a compile-time constant expression.

Answer : 05

Runtime Variable (int X = 10;): Allocated in RAM memory (e.g., address 0x7ffd9000). The CPU must fetch the value from physical RAM during execution.

Compile-time Constant (constexpr int Y = 10;): The compiler replaces the symbol 'Y' directly with the literal 10 in assembly code during compilation, eliminating runtime memory fetch overhead.

Answer : 06

Secondary Constants are structured, derived data types formed by grouping or referencing primary constants.

Derived/Compound Structures:

Arrays (std::array / C-style)

Strings (std::string / C-style)

Constant Pointers

Structures & Classes (struct / class)

Unions (union / std::variant)

Answer : 07

const int* ptr: Pointer to Constant Data. Read right-to-left: ptr is a pointer to an int that is const. The data value cannot be modified through ptr, but ptr can point to a new address.

int* const ptr: Constant Pointer. Read right-to-left: ptr is a const pointer to an int. The address stored in ptr cannot be changed, but the target value can be modified.

const int* const ptr: Constant Pointer to Constant Data. Neither the address nor the target value can be changed.

Answer : 08

In C-style strings, sequences of characters are stored in contiguous memory ending with a hidden null-terminator byte ('\0') that signals the end of the string.

Modern C++ recommends `std::string` and `std::string_view` for memory-safe and efficient string operations.

Answer : 09

struct Memory Layout: Members are allocated in contiguous memory blocks where each member has its own dedicated byte space.

union Memory Layout: All member variables share a single overlapping memory location equal to the size of its largest member.

Overwrite Behavior: Accessing a union member that was not the most recently written member results in undefined behavior or corrupted data.

Answer : 10

```
#include <iostream>
#include <string>
struct Student
{
    int age;
    float marks;
    char grade;
};
int main()
{
    // 1. Compile-time integer constant
    constexpr int maxScore = 100;

    // 2. String constant
    const std::string studentName = "Gaurav";

    // 3. Pointer to constant data
    int value = 50;
    const int* ptrToConst = &value;

    // 4. Struct constant
    const Student s1{20, 89.5f, 'A'};
```

```

std::cout << "Max Score: " << maxScore << std::endl;
std::cout << "Name: " << studentName << ", Grade: " << s1.grade << std::endl;
std::cout << "Pointer Value: " << *ptrToConst << std::endl;

return 0;
}

```

Answer : 11 Forgetting to initialize a const or constexpr variable upon declaration triggers an immediate compilation error.

Answer : 12 constexpr guarantees compile-time evaluation, enabling compiler optimizations and eliminating physical RAM fetch overhead at runtime.

Answer : 13 A constexpr function can be evaluated at runtime if its arguments are not compile-time constants, whereas a consteval function must evaluate at compile-time or compilation fails

Answer : 14 Reading from right to left: ptr is a const pointer to an int that is const (a constant pointer to constant data)

Answer : 15 Yes. const int* ptr prevents modifying value *through* ptr (*ptr = 20; fails), but value itself can still be modified directly if value is not declared const.

Answer : 16 The null-terminator is '\0', represented as a zero byte (0x00) in memory

Answer : 17 std::variant.

Answer : 18 std::array

Answer : 19 In a C++ struct, default member access is public, whereas in a class, default access is private.

Answer : 20 It embeds literal constant values directly into assembly instructions during compilation, preventing runtime memory fetch delays from RAM